Software Testing

Outline

- Overview of Software Testing
- Black Box / Functional Testing
- White Box / Structural Testing
- Integration Testing
- System Testing

White Box / Structural Testing

White Box Testing

- Also known as glass box, structural, clear box and open box testing.
- A software testing technique whereby explicit knowledge of the internal workings of the item being tested are used to select the test data.
- Unlike black box testing that is using the program specification to examine outputs, white box testing is based on specific knowledge of the source code to define the test cases and to examine outputs.

White Box Testing

The nature of white box testing allows for:

- Rigorous Definitions: In white box testing, testers have complete knowledge of the software's internal logic, including the code, algorithms, and data structures. This allows for a high level of specificity in defining what is to be tested and how.
- Mathematical Analysis: The inner workings of software can often be represented mathematically, using concepts like graphs, trees, and matrices. This allows testers to apply mathematical analysis methods for better understanding and assessment of the code.
- Useful Measurement: Since white box testing involves testing the internal structures of the application, it provides opportunities to gather detailed metrics about the code. These can include metrics related to code coverage, path coverage, and complexity. This data can be invaluable in identifying problem areas, optimizing performance, and improving overall code quality.

White Box Testing

- **Program Graph / Control Flow Graph**
  - DD-Paths
  - Test Coverage Metrics
  - Basis Path Testing
- Guidelines and Observations

Program Graphs

- Given a program written in an imperative programming language, its program graph is a directed graph in which nodes are statement fragments, and edges represent flow of control.
- A.k.a. Control Flow Graph
- If i and j are nodes in the program graph, an edge exists from node i to node j if and only if the statement fragment corresponding to node j can be executed immediately after the statement fragment corresponding to node i.

Program Graphs

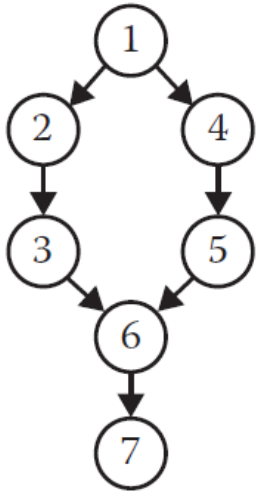
- In short:
  - In the context of white box testing, a program graph is used to visualize all possible execution paths in a piece of code. Each node in the graph represents program statement or statement fragment, and each directed edge represents the transition from one state to another.

Basic Structure of Program Graph

- 4 basic structures

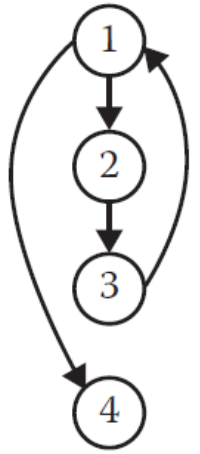
If–Then–Else

```
1  If <condition>
2     Then
3        <then statements>
4     Else
5        <else statements>
6  End If
7  <next statement>
```



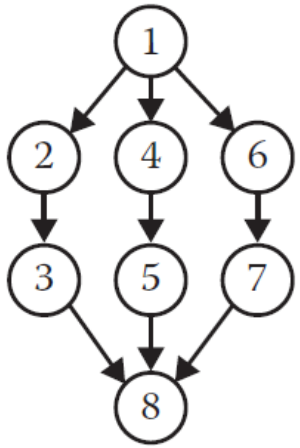
Pretest loop

```
1  While <condition>
2     <repeated body>
3  End While
4  <next statement>
```



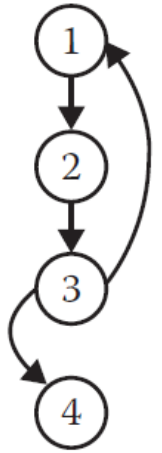
Case/Switch

```
1  Case n of 3
2     n=1:
3        <case 1  statements>
4     n=2:
5        <case 2  statements>
6     n=3:
7        <case 3  statements>
8  End Case
```



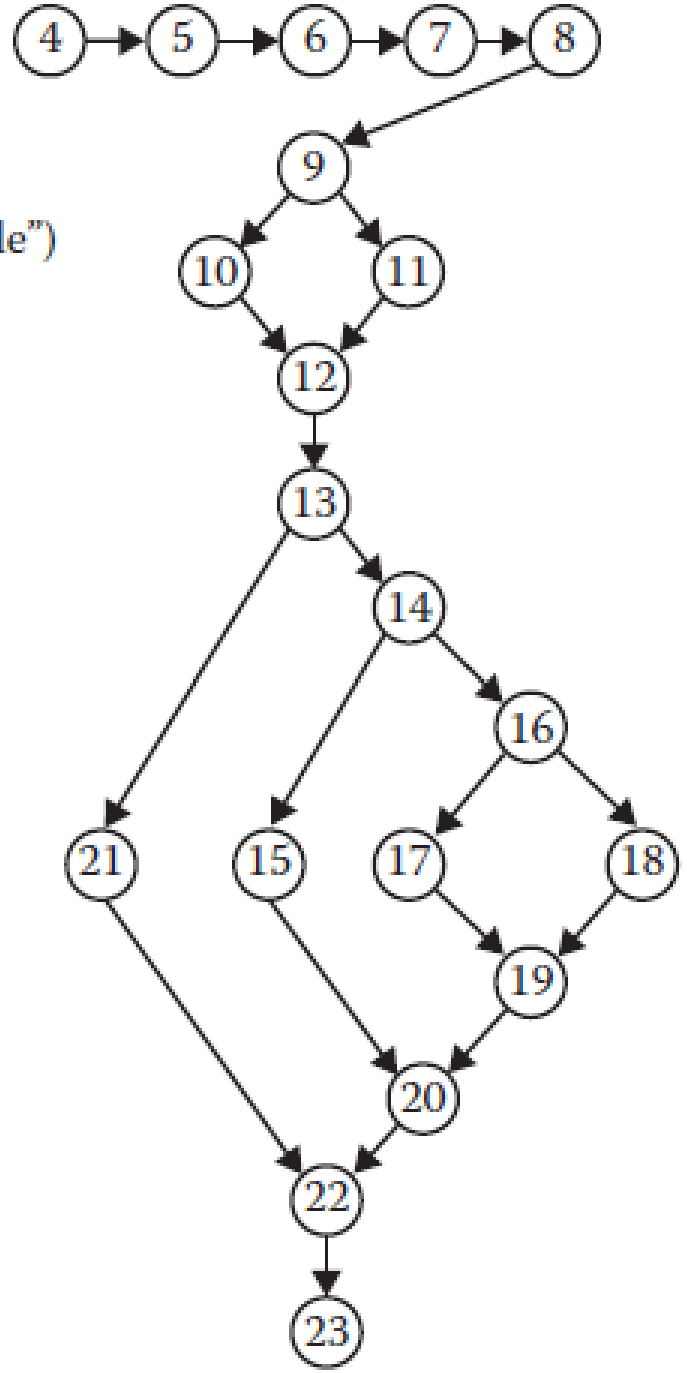
Posttest loop

```
1  Do
2     <repeated body>
3  Until <condition>
4  <next statement>
```



Example

```
1   Program triangle2
2   Dim a,b,c As Integer
3   Dim IsATrinagle As Boolean
4   Output("Enter 3 integers which are sides of a triangle")
5   Input(a,b,c)
6   Output("Side A is", a)
7   Output("Side B is", b)
8   Output("Side C is", c)
9   If (a < b + c) AND (b < a + c) AND (c < a + b)
10     Then IsATriangle = True
11     Else IsATriangle = False
12  EndIf
13  If IsATriangle
14     Then  If (a = b) AND (b = c)
15              Then Output ("Equilateral")
16              Else  If (a≠b) AND (a≠c) AND (b≠c)
17                       Then Output ("Scalene")
18                       Else Output ("Isosceles")
19                    EndIf
20              EndIf
21    Else  Output("Nota a Triangle")
22  EndIf
23  End triangle2
```



Example

- Non-executable statements are not included
- Nodes 4 to 8 are a sequence, nodes 9 to 12 are an if-then-else construct, nodes 13 to 22 are nested if-then-else construct.
- Node 4 is a program source that signify single-entry (indeg = 0)
- Node 23 is a sink node that signify single-exit (outdeg = 0)
- No loops exist, so this is a directed acyclic graph

White Box Testing

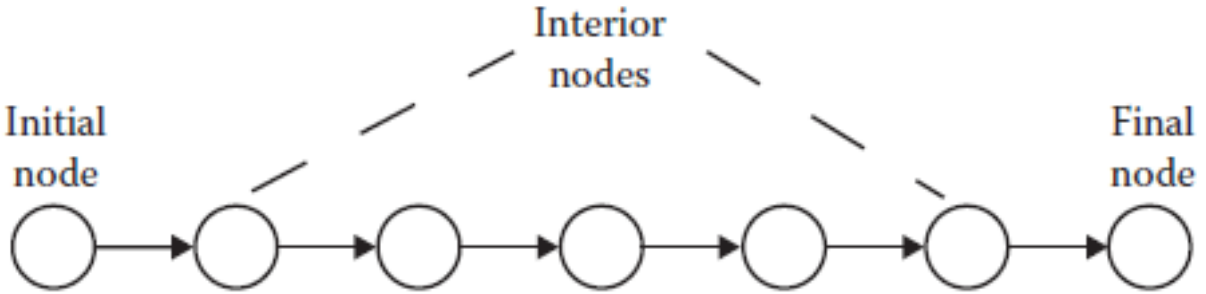
- Program Graph / Control Flow Graph
  - **DD-Paths**
  - Test Coverage Metrics
  - Basis Path Testing
- Guidelines and Observations

DD-Paths

- The best known form of structural testing is based on a construct known as a decision-to-decision path.
- A DD-Path is a chain obtained from a program graph. It is a sub-path of the Control Flow Graph.
- It is the path from one decision node to another, or from a decision node to a terminal node

DD-Paths

- More formally a DD-Path is a chain obtained from a program graph such that:
  - Case1: it consists of a single node with indeg=0.
  - Case2: it consists of a single node with outdeg=0,
  - Case3: it consists of a single node with indeg ≥ 2 or outdeg ≥ 2 (for example, node D and A)
  - Case4: it consists of a single node with indeg =1, and outdeg = 1
  - Case5: it is a maximal chain of length ≥ 1



DD-Path Graph

- Given a program written in an imperative language, its DD-path graph is the directed graph in which nodes are DD-paths of its program graph, and edges represent control flow between successor DD-paths.

CFG to DD-Path Graph

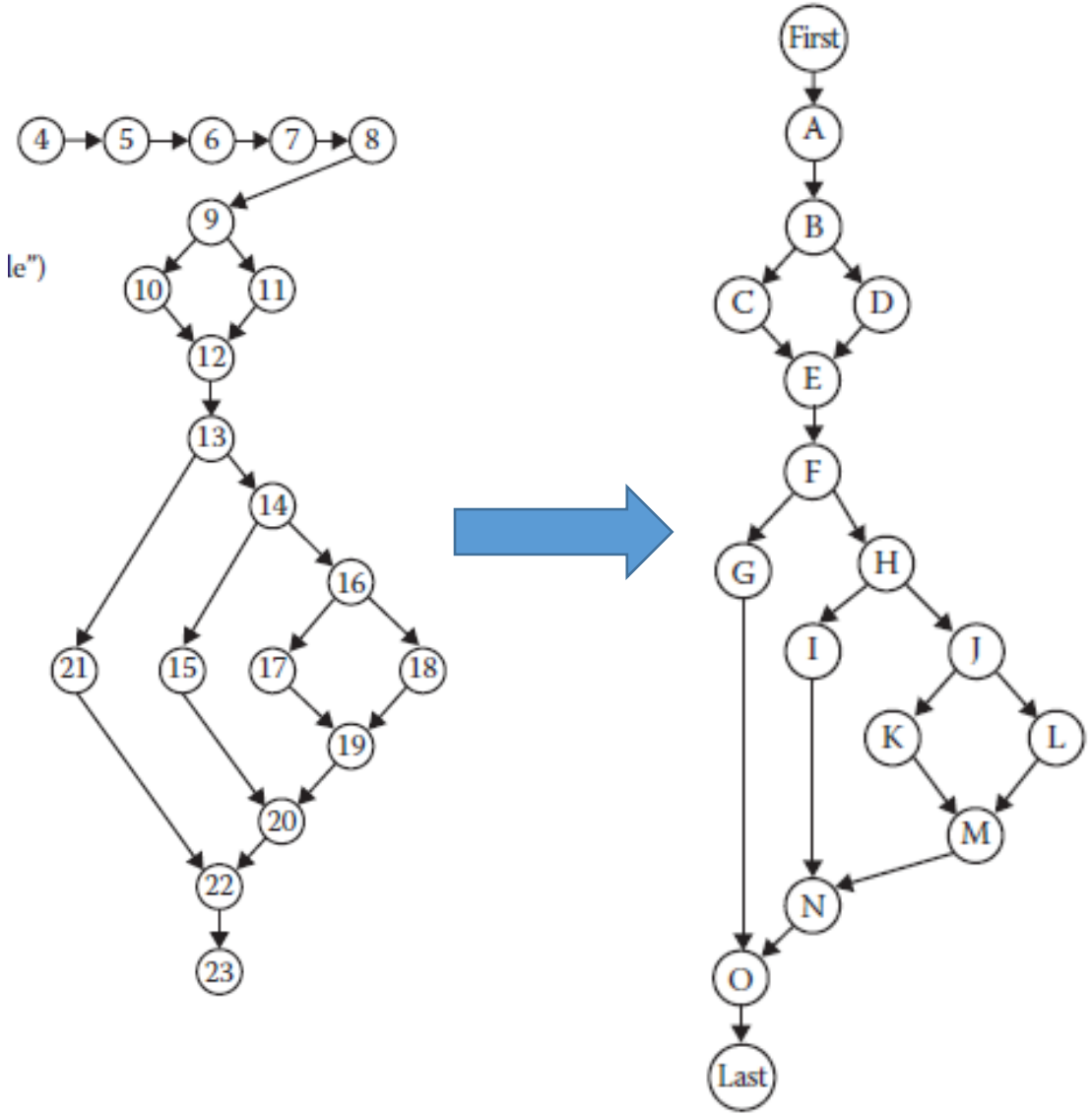


Figure 8.2 Nodes	DD-Path	Case of definition
4	First	1
5-8	A	5
9	B	3
10	C	4
11	D	4
12	E	3
13	F	3
14	H	3
15	I	4
16	J	3
17	K	4
18	L	4
19	M	3
20	N	3
21	G	4
22	O	3
23	Last	2

White Box Testing

- Program Graph / Control Flow Graph
  - DD-Paths
  - **Test Coverage Metrics**
  - Basis Path Testing
- Guidelines and Observations

Test Coverage Metrics

- The motivation of using DD-paths is that they enable very precise descriptions of test coverage.
- The fundamental limitations of specification-based testing is that it is impossible to know either the extent of redundancy or the possibility of gaps corresponding to the way a set of functional test cases exercises a program
- Test coverage metrics are a device to measure the extend to which a set of test cases covers a program.

E.F. Miller's Test Coverage Metrics

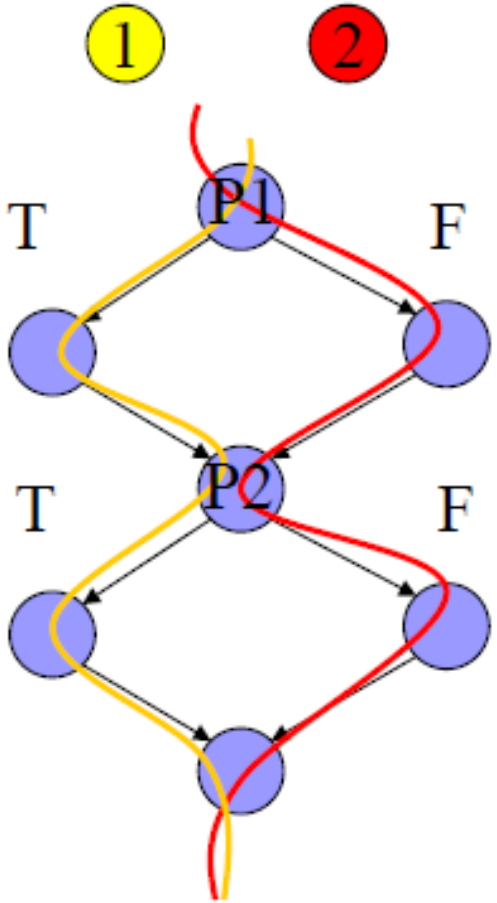
Metric	*Description of Coverage*
C_0	Every statement
C_1	Every DD-path
C_{1p}	Every predicate to each outcome
C_2	C_1 coverage + loop coverage
C_d	C_1 coverage + every dependent pair of DD-paths
C_{MCC}	Multiple condition coverage
C_{ik}	Every program path that contains up to k repetitions of a loop (usually $k = 2$)
C_{stat}	"Statistically significant" fraction of paths
C_∞	All possible execution paths

Statement and Predicate Coverage Testing

- Statement coverage based testing aims to devise test cases that collectively exercise all statements in a program.
- Predicate coverage (or branch coverage, or decision coverage) based testing aims to devise test cases that evaluate each simple predicate of the program to True and False. Here the term simple predicate refers to either a single predicate or a compound Boolean expression that is considered as a single unit that evaluates to True or False. This amounts to traversing every edge in the DD-Path graph.
- For example in predicate coverage for the condition
  - *if(A or B) then C* we could consider the test cases *A=True, B= False* (true case), and *A=False, B=False* (false case).

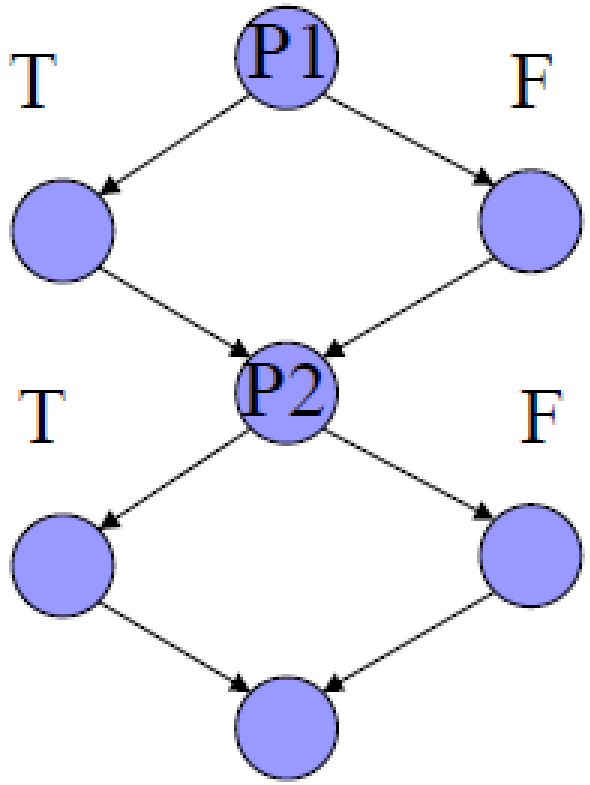
DD-Path Graph Edge Coverage C_1

- Here a T,T and F,F combination will suffice to have DD-Path Graph edge coverage or Predicate coverage C_1



DD-Path Coverage Testing C_{1P}

- This is the same as the C_1 but now we must consider test cases that exercise all possible outcomes of the choices T,T, T,F, F,T, F,F for the predicates P1, and P2 respectively, in the DD-Path graph.



Multiple Condition Coverage Testing

- Now if we consider that the predicates P1 is a compound predicate (i.e. (A or B)) then Multiple Condition Coverage Testing requires that each possible combination of inputs be tested for each decision.
- Example: “if (A or B)” requires 4 test cases:
  - A = True, B = True
  - A = True, B = False
  - A = False, B = True
  - A = False, B = False
- The problem: For n conditions, 2^n test cases are needed, and this grows exponentially with n

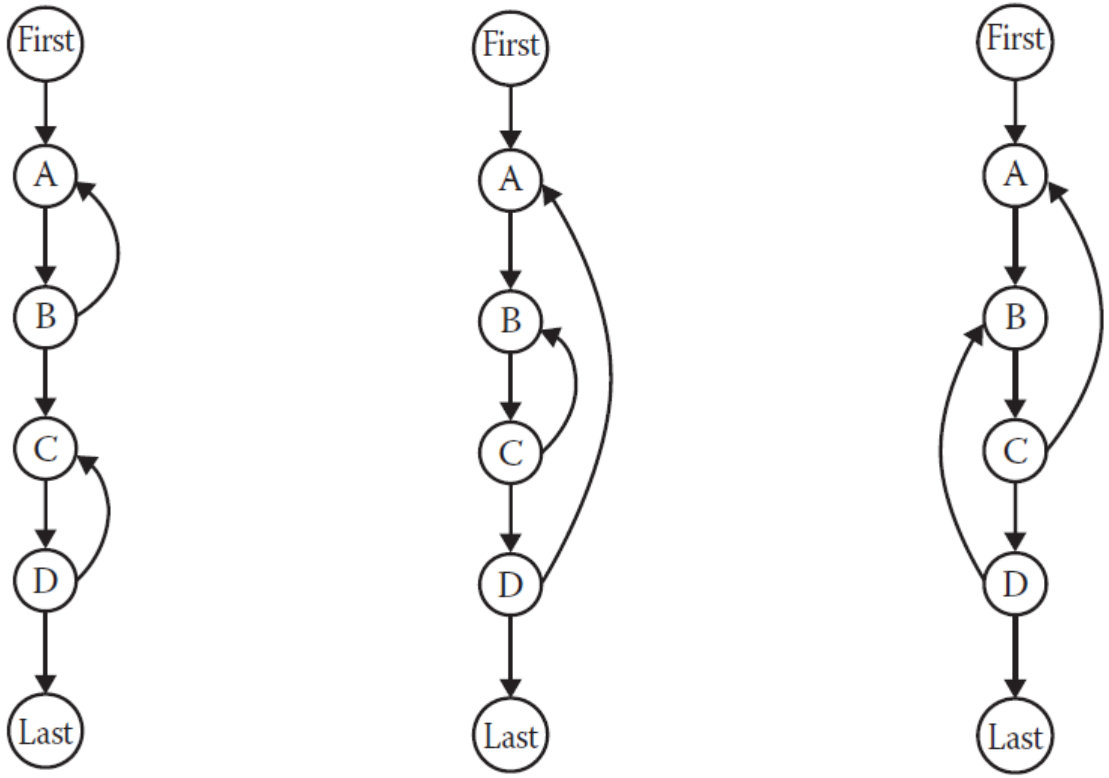
Dependent DD-Path Pairs Coverage Testing C_d

- In simple C_1 coverage criterion we are interested simply to traverse all edges in the DD-Path graph.
- If we enhance this coverage criterion by ensuring that we traverse dependent pairs of DD-Paths also we may have the chance of revealing more errors that are based on data flow dependencies.

Dependent DD-Path Pairs Coverage Testing C_d

- More specifically, two DD-Paths are said to be dependent iff there is a define/reference relationship between these DD-Paths, in which a variable is defined (receives a value) in one DD-Path and is referenced in the other.
- In C_d testing we are interested on covering all edges of the DD-Path graph and all dependent DD-Path pairs.
- The importance of these dependencies is that they are closely related to the problem of infeasible paths. We have good examples of dependent pairs of DD-paths: in slide 16, C and H are such a pair, as are DD-paths D and H.

Loop Coverage



Concatenated Loop, Nested Loop, and Knotted Loop

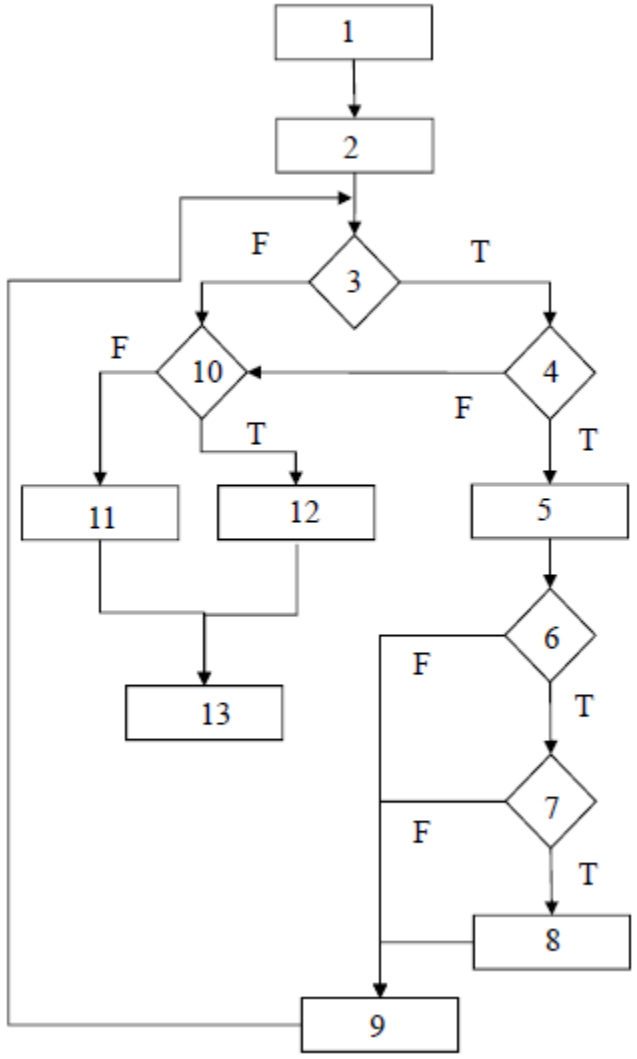
Loop Coverage

- The simple view of loop testing coverage is that we must devise test cases that exercise the two possible outcomes of the decision of a loop condition that is one to traverse the loop and the other to exit (or not enter) the loop.
- An extension would be to consider a modified boundary value analysis approach where the loop index is given a minimum, minimum +, a nominal, a maximum -, and a maximum value or even robustness testing.

Loop Coverage

- In the case of nested loops we start with the inner most loop and we proceed outwards.
- If loops are knotted then we must apply data flow analysis testing techniques, that we will examine later in the module.

Examples



Statement Coverage C_0:

- SCPath1: 1-2-3(F)-10(F)-11-13
- SCPath2: 1-2-3(T)-4(T)-5-6(T)-7(T)-8-9-3(F)-10(T)-12-13

Branch or Decision Coverage C_1:

- BCPath1: 1-2-3(F)-10(F)-11-13
- BCPath2: 1-2-3(T)-4(T)-5-6(T)-7(T)-8-9-3(F)-10(T)-12-13
- BCPath3: 1-2-3(T)-4(F)-10(F)-11-13
- BCPath4: 1-2-3(T)-4(T)-5-6(F)-9-3(F)-10(F)-11-13
- BCPath5: 1-2-3(T)-4(T)-5-6(T)-7(F)-9-3(F)-10(F)-11-13

White Box Testing

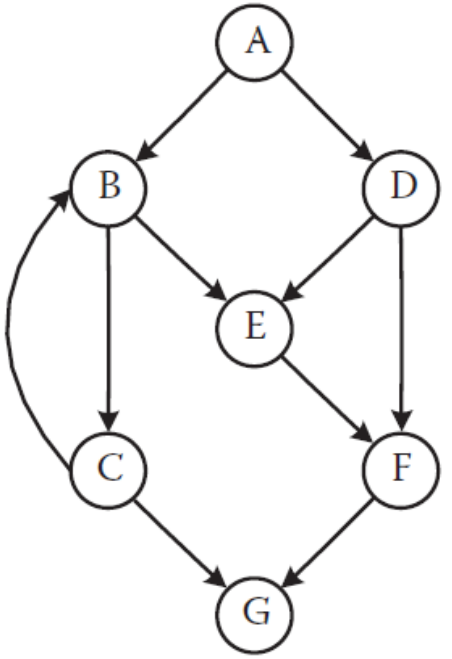
- Program Graph / Control Flow Graph
  - DD-Paths
  - Test Coverage Metrics
  - **Basis Path Testing**
- Guidelines and Observations

McCabe's Basis Path Testing

- Basis Path Testing is a white box testing approach proposed by Tom McCabe. The idea is to derive a measure of the logical complexity of a program and use this as a guide for defining a basic set of execution paths to test.
- The measure of complexity used is typically the cyclomatic complexity, which is based on the control flow structure of the program.
- The method to carry out basis path testing has four steps: -
  1. Compute the program graph
  2. Calculate the cyclomatic complexity
  3. Select a basis set of paths
  4. Generate test cases for each of these paths.

McCabe's Basis Path Method

- Step 1: To begin, we need a program graph from which to construct a basis
- The below figure is a directed graph which is the DD-Path graph of some program.
- The program has a single entry (A) and a single exit (G).



McCabe’s Basis Path Method

- Step 2: The cyclomatic complexity of a program, according to McCabe's method, is calculated as the number of linearly independent paths through the program's source code.
- Essentially, it provides a count of the number of decisions in the source code to give an indication of the program's overall complexity.

McCabe's Basis Path Method

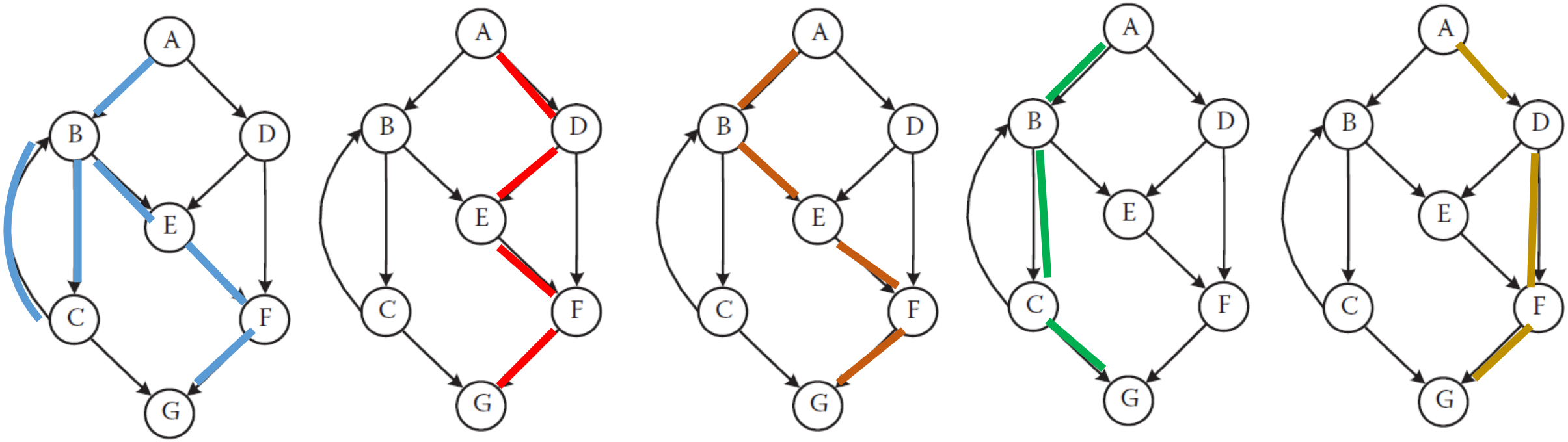
- The formula for cyclomatic complexity is given by
  - $V(G) = e - n + 2p$ or $V(G) = e - n + p$
  - Where, e: the number edges; n: the number of nodes; p: number of connected regions.
- The number of linearly independent paths from source node to sink node in the graph is
  - $V(G) = e - n + 2p = 10 - 7 + 2(1) = 5$
- The number of linearly independent circuits in the graph is
  - $V(G) = e - n + p = 11 - 7 + 1 = 5$

McCabe's Basis Path Method

- Step 3: A linearly independent path is any path through the source code that introduces at least one new set of processing statements or a new condition.
- McCabe's Baseline Method
  - The method begins with the selection of a "baseline path", which should correspond to a normal execution of a program (from start node to end node, and has as many decisions as possible).
  - The algorithm proceeds by retracing the paths visited and flipping the conditions one at a time.
  - The process repeats up to the point all flips have been considered.

McCabe’s Basis Path Method

- McCabe’s Baseline Method

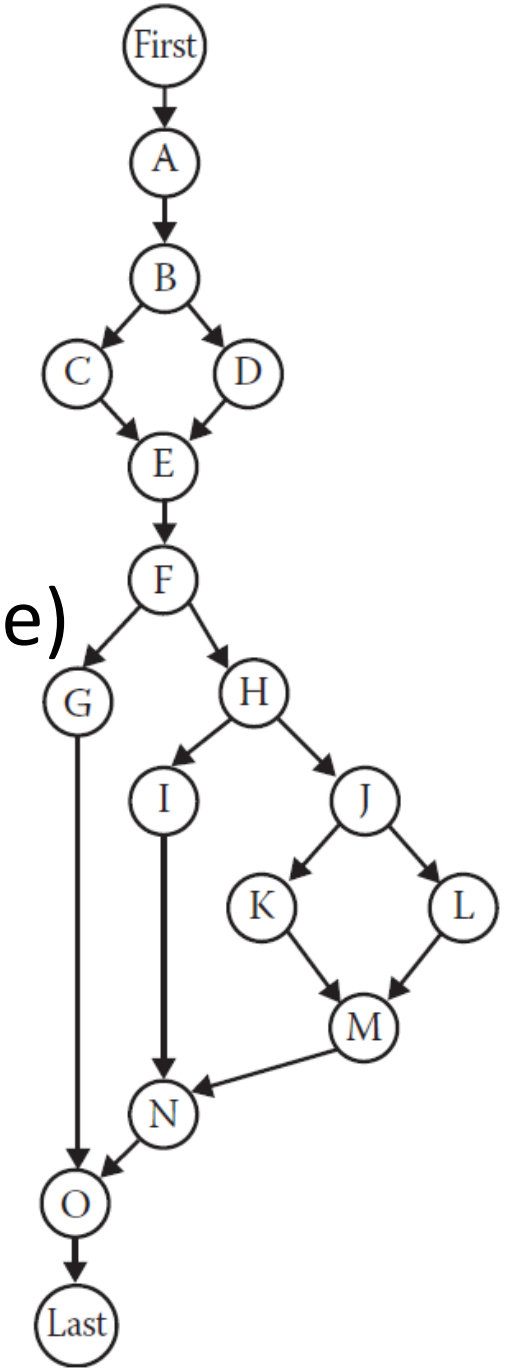


Observation on Basis Path Method

- Two major soft spots occur in McCabe's view:
  - Testing the set of basis paths is sufficient (it is not!)
  - Have to make program paths look like a vector space.

Observation on Basis Path Method

- V(G)=e-n+2p =18-15+2(1) =5
  - Original p1: A-B-C-E-F-H-J-K-M-N-O-Last (Scalene)
  - Flip p1 at B - p2: A-B-D-E-F-H-J-K-M-N-O-Last (Infeasible)
  - Flip p1 at F - p3: A-B-C-E-F-G-O-Last (Infeasible)
  - Flip p1 at H - p4: A-B-C-E-F-H-I-N-O-Last (Equilateral)
  - Flip p1 at J - p5:A-B-C-E-F-H-J-L-M-N-O-Last (Isosceles)



Observation on Basis Path Method

- McCabe’s procedure successfully identifies basis path that are topologically independent, but when these contradict semantic dependences, topologically possible paths are seen to be logically infeasible.

Observation on Basis Path Method

- If we assume that the logic of the program dictates that “if node C is traversed, then we must traverse node H, and if node D is traversed, then we must traverse Node G”
  - These constraints will eliminate Paths 2,3
  - We also need a basis path for the NotATriangle case
  - We are left to consider four feasible paths:
    - p1: A-B-C-E-F-H-J-K-M-N-O-Last (Scalene)
    - p4: A-B-C-E-F-H-I-N-O-Last (Equilateral)
    - p5:A-B-C-E-F-H-J-L-M-N-O-Last (Isosceles)
    - p6: A-B-D-E-F-G-O-Last (Not a triangle)

White Box Testing

- Program Graph / Control Flow Graph
  - DD-Paths
  - Test Coverage Metrics
  - Basis Path Testing
- **Guidelines and Observations**

Guidelines and Observations

- Functional testing focuses on properties that are “too far” or disassociated from the source code being tested
- Path testing focuses too much on the graph and not the logic of the code, but is very useful to measure the quality of the testing (coverage criteria)
- Basis path testing is giving us a lower bound of how much testing is necessary

Guidelines and Observations

- Path testing is providing a set of metrics that act as cross checks to functional testing
  - When a program path is traversed by several functional test cases we suspect redundancy
  - When we fail to attain DD-Path graph coverage, we know that there are gaps in the functional test cases
- Coverage metrics can be used as
  - Setting minimum testing standards and as mechanism to selectively test portions of the code more rigorously than others

End